

BOOK 18 — JAVA DESIGN PATTERNS

Creational, Structural & Behavioral Patterns — Refactoring a Bad-Design App, Pattern by Pattern (Telugu + English)

Series: Java Interview + Development Mastery Series **Book Number:** 18 of 24 (+1 Special: Book 15A)
Versions Covered: Java 8/17/21 (patterns shown with modern Java idioms — records, sealed types, lambdas where they simplify classic GoF pattern code) **Prerequisites:** Books 01–07 (OOP, SOLID, Generics, Java 8+ lambdas); connections drawn throughout to Books 09–17 (Spring, Spring Boot, Microservices, Kafka) **Next Book:** 19_Low_Level_Design.md

★ **RECRUITER-PRIORITY NOTE:** Design patterns interviews for a "Java Full Stack Developer" role rarely ask "define Singleton" in isolation — they ask "where have you seen this in Spring Boot?" This book deliberately traces every pattern back into the Spring/Spring Boot/Microservices/Kafka machinery already built in Books 09–17, so pattern knowledge reinforces, rather than sits apart from, the recruiter-priority stack.

How to Use This Book / ఈ పుస్తకాన్ని ఎలా ఉపయోగించాలి

Telugu: ఈ పుస్తకం అంతటా ఒకే **bad-design mini app** — `OrderProcessingSystem` (OPS) — వాడతాము. ఇది ఒక god-class, hardcoded dependencies, giant if-else chains, tightly-coupled notification logic తో నిండి ఉంటుంది — real legacy code లో కనిపించే విధంగానే. ప్రతి chapter ఒక design pattern నేర్పి, OPS లోని ఒక్క నిర్దిష్ట సమస్యని ఆ pattern తో refactor చేస్తుంది — చివరికి Chapter 16 నాటికి, OPS పూర్తిగా, pattern-by-pattern, clean architecture గా మారుతుంది.

English: Throughout this book we use one shared **bad-design mini app** — `OrderProcessingSystem` (OPS). It's a god-class, riddled with hardcoded dependencies, giant if-else chains, and tightly-coupled notification logic — exactly like real legacy code. Each chapter teaches one design pattern and refactors one specific problem in OPS using that pattern — by Chapter 16, OPS has been transformed, pattern by pattern, into clean architecture.

The Starting Point — OrderProcessingSystem (Intentionally Bad Design)

```
// THIS IS THE BAD-DESIGN STARTING POINT - we refactor pieces of this, chapter by chapter.
class OrderProcessingSystem {
    static OrderProcessingSystem instance = new OrderProcessingSystem(); // ad-hoc, unsafe
    "singleton" (Ch.1 fixes this)

    void processOrder(String type, String customerName, String email, String phone,
        double amount, String paymentType, String status) { // telescoping
params (Ch.4 fixes this)
        Object notifier;
        if (type.equals("EMAIL")) notifier = new EmailSender(); // hardcoded
`new` (Ch.2 fixes this)
        else notifier = new SmsSender();

        if (paymentType.equals("CARD")) { /* card logic inline */ } // if-else
dispatch (Ch.12 fixes this)
        else if (paymentType.equals("UPI")) { /* upi logic inline */ }

        if (status.equals("PLACED")) { /* ... */ } // status if-else
(Ch.15 fixes this)
        else if (status.equals("SHIPPED")) { /* ... */ }
        // ... validation, logging, and persistence all inline here too (Ch.16 fixes this)
    }
}
```

Learning Objectives

1. Recognize the design problems that each Creational, Structural, and Behavioral pattern solves.
2. Implement all 16 classic GoF patterns using modern Java idioms.
3. Identify each pattern already at work inside Spring, Spring Boot, and Kafka.
4. Refactor a real bad-design codebase incrementally, pattern by pattern.
5. Know when NOT to apply a pattern (over-engineering awareness).

Table of Contents

Ch.	Title	Category
1	Singleton Pattern	Creational
2	Factory Method Pattern	Creational
3	Abstract Factory Pattern	Creational
4	Builder Pattern	Creational
5	Prototype Pattern	Creational
6	Adapter Pattern	Structural
7	Decorator Pattern	Structural
8	Facade Pattern	Structural

Ch.	Title	Category
9	Proxy Pattern	Structural
10	Composite Pattern	Structural
11	Observer Pattern	Behavioral
12	Strategy Pattern	Behavioral
13	Template Method Pattern	Behavioral
14	Command Pattern	Behavioral
15	State Pattern	Behavioral
16	Chain of Responsibility Pattern	Behavioral
—	Final Revision Notes, Cheat Sheet, Interview Bank, Revision Plans, Mastery Checklist	

CHAPTER 1 — Singleton Pattern

Telugu Explanation

OPS లో `static instance = new OrderProcessingSystem()` అనేది class-load సమయంలోనే create అవుతుంది — lazy కాదు, thread-safety ని guarantee చేయదు (concurrent access లో double-instantiation risk ఉండొచ్చు complex init logic ఉంటే). **Singleton pattern** ఒక class కి exactly ఒక్క instance మాత్రమే ఉండేలా, మరియు దాన్ని globally access చేసేలా, thread-safe గా guarantee చేస్తుంది.

Professional English Explanation

OPS's `static instance = new OrderProcessingSystem()` is created eagerly at class-load time — not lazy, and doesn't explicitly guarantee thread safety for more complex initialization logic. The **Singleton pattern** formally guarantees a class has exactly one instance, provides global access to it, and does so thread-safely.

Java Code — Thread-Safe Singleton (Enum-Based, the Modern Recommended Approach)

```
public enum ConfigurationManager {                                // Book 07 - enums are inherently
    thread-safe singletons
    INSTANCE;

    private final Map<String, String> settings = new ConcurrentHashMap<>(); // Book 08 -
    thread-safe collection

    public String get(String key) { return settings.get(key); }
    public void set(String key, String value) { settings.put(key, value); }
}

// Usage: ConfigurationManager.INSTANCE.get("tax.rate");
```

Internal Working

- The **enum singleton** approach (Joshua Bloch's recommended idiom) is preferred over the classic `private static volatile instance` + double-checked locking pattern because the JVM guarantees enum instance creation is inherently thread-safe and serialization-safe (a classic lazy singleton can be broken by reflection or deserialization unless explicitly guarded against).
- **Spring Boot connection:** every `@Service` / `@Component` / `@Repository` bean (Book 11, Ch.2) is a **singleton by default** within its `ApplicationContext` — Spring manages this via its bean scope mechanism rather than the GoF Singleton pattern's static-instance trick, but the *goal* (exactly one shared instance) is identical; this is why understanding Singleton directly explains Spring's default bean scope.
- Singleton is also one of the most **overused** patterns — global mutable state accessed from anywhere makes testing hard (Book 15's mockability principles) and hides dependencies that should be explicit (Book 11's DI philosophy exists partly to avoid ad-hoc singletons like OPS's).

Real-World Example

A JDBC `DataSource` /connection pool (Book 09) is effectively a singleton per application — you don't want dozens of independently-created connection pools competing for database connections.

Interview Answer

"Singleton guarantees exactly one instance of a class, accessed globally. The enum-based implementation is the modern recommended approach since the JVM guarantees it's thread-safe and immune to reflection/serialization attacks that can break a naive lazy singleton. Spring Boot beans are singletons by default within the `ApplicationContext`, achieved through the container's bean scope management rather than a static instance field — but conceptually it's the same guarantee. Singleton is also commonly overused: global mutable state makes testing harder and hides dependencies, which is part of why Spring favors dependency injection over manual singletons."

Cross Questions

- Q: Why is the enum-based Singleton preferred over double-checked locking with a `volatile` field? → A: The JVM guarantees enum instantiation is thread-safe and it's immune to reflection and serialization attacks that can otherwise create a second instance of a naive lazy singleton.
- Q: Are Spring `@Service` beans Singletons in the GoF pattern sense? → A: Conceptually yes (exactly one shared instance), but implemented via the container's bean-scope management, not the static-instance-field mechanism.

Tricky Questions

- Q: Is Singleton always a good choice for shared state? → A: No — it introduces global mutable state, hides dependencies, and complicates unit testing; Spring's DI-managed singletons are preferred precisely because dependencies stay explicit and mockable (Book 15).

Coding Exercise

L1: Implement `ConfigurationManager` as an enum singleton. **L2:** Attempt to break a naive (non-enum) lazy singleton via reflection, then explain why the enum version resists it. **L3:** Compare OPS's `static instance = new OrderProcessingSystem()` to the enum singleton. **L4 (Interview):** Explain why Spring beans are singletons by default and how that relates to this pattern. **L5 (Senior — OPS Refactor):** Extract OPS's ad-hoc static instance into a properly designed singleton or, better, argue for replacing it with Spring DI entirely. **L6 (Mastery):** Explain when Singleton is an anti-pattern vs a legitimate design choice.

CHAPTER 2 — Factory Method Pattern

Telugu Explanation

OPS లో `new EmailSender()` / `new SmsSender()` ని if-else తో directly instantiate చేసాము — ఇది Open/Closed Principle (Book 02) ని violate చేస్తుంది: కొత్త notification type (Push) add చేయాలంటే ఈ if-else block ని modify చేయాలి. **Factory Method** object creation ని ఒక dedicated method/class కి delegate చేస్తుంది, calling code ని concrete class నుండి decouple చేస్తుంది.

Professional English Explanation

OPS directly instantiated `new EmailSender()` / `new SmsSender()` via if-else — this violates the Open/Closed Principle (Book 02): adding a new notification type (Push) requires modifying this if-else block. **Factory Method** delegates object creation to a dedicated method/class, decoupling calling code from concrete classes.

Java Code — Factory Method Replacing OPS's If-Else Instantiation

```
interface Notifier { void send(String message, String recipient); }

class EmailNotifier implements Notifier {
    public void send(String message, String recipient) { /* SMTP logic */ }
}
class SmsNotifier implements Notifier {
    public void send(String message, String recipient) { /* SMS gateway logic */ }
}

class NotifierFactory {
    static Notifier create(NotificationType type) {
        return switch (type) {
            case EMAIL -> new EmailNotifier();
            case SMS -> new SmsNotifier();
            case PUSH -> new PushNotifier();
        };
    }
}

// the Factory Method
// Book 07 - Java 17
// adding PUSH needs only
// ONE new branch here,
// OPS's original if-else is now gone entirely

// OPS's processOrder now does: Notifier notifier = NotifierFactory.create(type);
```

Internal Working

- The calling code (`OrderProcessingSystem`) now depends only on the `Notifier` **interface** (Book 02's polymorphism, Book 06's contract-based design) — it has zero knowledge of `EmailNotifier` / `SmsNotifier` concrete classes, which is exactly the Dependency Inversion half of SOLID (Book 02) in action.
- **Spring Boot connection:** Spring's `BeanFactory` / `ApplicationContext` (Book 11, Ch.2) is itself a giant, configurable Factory — `context.getBean(PaymentGateway.class)` returns the correct

concrete implementation without the caller ever writing `new`, exactly like

`NotifierFactory.create()` above, but driven by configuration/annotations instead of a `switch`.

- Factory Method centralizes creation logic in **one place** — if `EmailNotifier`'s constructor later needs an SMTP config object, only the factory changes, not every call site that used to say `new EmailSender()`.

Real-World Example

`DocumentBuilderFactory.newInstance()` and `DateTimeFormatter.ofPattern(...)` in the JDK itself are Factory Method-style APIs — callers never construct the underlying implementation classes directly.

Interview Answer

"Factory Method centralizes object creation behind a method or class, so calling code depends only on an interface, not concrete classes — this satisfies the Open/Closed Principle, since adding a new type means adding a new factory branch, not modifying every call site. Spring's

`BeanFactory` / `ApplicationContext` is a configurable, industrial-strength version of exactly this pattern — `getBean()` returns the correct implementation without the caller ever writing `new`."

Cross Questions

- Q: What SOLID principle does Factory Method directly support? → A: Open/Closed — new types can be added by extending the factory, without modifying existing call sites that use the interface.
- Q: How does Spring's `ApplicationContext` relate to Factory Method? → A: It's a large-scale, configuration-driven implementation of the same idea — returning correct concrete bean instances behind an interface/type request, without the caller writing `new`.

Tricky Questions

- Q: Does using a `switch` in a factory method still leave an if-else-like maintenance problem? → A: Somewhat — it's a single centralized point of change instead of scattered call sites, which is the real win; for highly dynamic type sets, combining Factory Method with Spring's DI container (auto-wiring by type/qualifier) removes even that switch.

Coding Exercise

L1: Implement `NotifierFactory` replacing OPS's if-else with a `switch`. **L2:** Add a `PushNotifier` type and confirm no existing code needed modification besides the factory. **L3:** Refactor `NotifierFactory` to be Spring-managed, injecting all `Notifier` beans by type. **L4 (Interview):** Explain Factory Method and its relationship to Open/Closed. **L5 (Senior — OPS Refactor):** Fully remove OPS's `if (type.equals("EMAIL"))` block using this factory. **L6 (Mastery):** Compare Factory Method to Spring's `@Qualifier`-based bean resolution.

CHAPTER 3 — Abstract Factory Pattern

Telugu Explanation

Factory Method ఒక్క product create చేస్తుంది. **Abstract Factory** ఒక **family of related products** ని create చేస్తుంది — ఉదా: different payment providers (Stripe vs Razorpay) కి వేర్వేరు `PaymentGateway` + `RefundProcessor` + `PaymentValidator` combination అవసరం అయితే, ఒక్క factory అన్నింటినీ consistent గా ఇస్తుంది, mismatched combinations (Stripe gateway + Razorpay validator) రాకుండా చూస్తుంది.

Professional English Explanation

Factory Method creates one product. **Abstract Factory** creates a **family of related products** — for example, if different payment providers (Stripe vs Razorpay) each need a matching `PaymentGateway` + `RefundProcessor` + `PaymentValidator`, one factory provides all of them consistently, preventing mismatched combinations (a Stripe gateway paired with a Razorpay validator).

Java Code — Abstract Factory for Payment Provider Families

```
interface PaymentProviderFactory {
    PaymentGateway createGateway();
    RefundProcessor createRefundProcessor();
}

class StripeProviderFactory implements PaymentProviderFactory {
    public PaymentGateway createGateway() { return new StripeGateway(); }
    public RefundProcessor createRefundProcessor() { return new StripeRefundProcessor(); } //
    guaranteed matching family
}

class RazorpayProviderFactory implements PaymentProviderFactory {
    public PaymentGateway createGateway() { return new RazorpayGateway(); }
    public RefundProcessor createRefundProcessor() { return new RazorpayRefundProcessor(); }
}

// Usage: PaymentProviderFactory factory = config.isStripe() ? new StripeProviderFactory() :
// new RazorpayProviderFactory();
// factory.createGateway() and factory.createRefundProcessor() are ALWAYS from the same
// provider - never mismatched.
```

Internal Working

- The key difference from Ch.2's Factory Method: Abstract Factory's interface has **multiple creation methods**, one per product in the family, and every concrete factory implementation guarantees all of them belong to the same consistent family — this is precisely what prevents a caller from accidentally wiring a `StripeGateway` with a `RazorpayRefundProcessor`.
- Spring Boot connection:** Spring profiles combined with `@Configuration` classes (Book 12, Ch.8) often implement this exact pattern — a `StripeConfig` `@Configuration` class defines all Stripe-related beans together, activated only under the `stripe` profile, guaranteeing the whole family is wired consistently, never mixed with `RazorpayConfig`'s beans.

- Abstract Factory adds real complexity — it's justified only when there are genuinely multiple **families** of related objects; for a single product type (Ch.2's notifiers), plain Factory Method is simpler and sufficient.

Real-World Example

JDBC driver implementations are conceptually an Abstract Factory family — a `Connection`, `Statement`, and `ResultSet` from the same JDBC driver are guaranteed to work together correctly; mixing a MySQL `Connection` with a PostgreSQL `Statement` implementation isn't possible because each driver's factory produces its own consistent family.

Interview Answer

"Abstract Factory creates a family of related objects through one factory interface with multiple creation methods, guaranteeing the returned objects are always from the same consistent family — unlike Factory Method, which creates a single product. In Spring Boot, profile-scoped `@Configuration` classes often implement this: a `StripeConfig` class wires all Stripe-related beans together under one profile, preventing them from being mixed with a different provider's beans."

Cross Questions

- Q: What's the structural difference between Factory Method and Abstract Factory? → A: Factory Method creates one product via one method; Abstract Factory creates a family of related products via multiple methods on one factory interface, guaranteeing consistency across the family.
- Q: How do Spring profile-scoped `@Configuration` classes resemble Abstract Factory? → A: Each profile's configuration class wires a full, mutually-consistent set of beans (a "family"), activated together, preventing beans from different families being mixed.

Tricky Questions

- Q: Should Abstract Factory replace Ch.2's Factory Method everywhere for consistency? → A: No — it's justified only when there are genuinely multiple families of related products; applying it to a single-product scenario (like OPS's notifiers) adds unnecessary interfaces and complexity for no benefit.

Coding Exercise

L1: Implement `PaymentProviderFactory` with Stripe and Razorpay families. **L2:** Add a third product (`PaymentValidator`) to the family and update both concrete factories. **L3:** Implement the Spring profile-scoped `@Configuration` equivalent for one provider family. **L4 (Interview):** Explain Abstract Factory and how it differs from Factory Method. **L5 (Senior):** Identify a place in a hypothetical system where Abstract Factory would be over-engineering, and justify using plain Factory Method instead. **L6 (Mastery):** Design an Abstract Factory family for supporting three different cloud storage providers (S3, GCS, Azure Blob).

CHAPTER 4 — Builder Pattern

Telugu Explanation

OPS లోని `processOrder(type, name, email, phone, amount, paymentType, status)` ఒక **telescoping constructor/method** anti-pattern — ఎన్ని parameters ఉన్నాయో, ఏ order లో ఉన్నాయో గుర్తుంచుకోవడం కష్టం, మరియు రెండు `String` params (email, phone) accidentally swap అయితే compiler కూడా పట్టుకోదు. **Builder pattern** step-by-step, named, readable object construction ఇస్తుంది.

Professional English Explanation

OPS's `processOrder(type, name, email, phone, amount, paymentType, status)` is a **telescoping parameter list** anti-pattern — hard to remember how many parameters there are or their order, and if two `String` params (email, phone) get accidentally swapped, the compiler won't catch it. The **Builder pattern** gives step-by-step, named, readable object construction.

Java Code — Builder Replacing OPS's Telescoping Parameters

```
class OrderRequest {
    private final String customerName;
    private final String email;
    private final String phone;
    private final BigDecimal amount;
    private final PaymentType paymentType;

    private OrderRequest(Builder b) { // private constructor - only
Builder can create this
        this.customerName = b.customerName; this.email = b.email;
        this.phone = b.phone; this.amount = b.amount; this.paymentType = b.paymentType;
    }

    static Builder builder() { return new Builder(); }

    static class Builder {
        private String customerName, email, phone;
        private BigDecimal amount;
        private PaymentType paymentType;

        Builder customerName(String v) { this.customerName = v; return this; } // fluent,
self-returning (method chaining)
        Builder email(String v) { this.email = v; return this; }
        Builder phone(String v) { this.phone = v; return this; }
        Builder amount(BigDecimal v) { this.amount = v; return this; }
        Builder paymentType(PaymentType v) { this.paymentType = v; return this; }

        OrderRequest build() {
            Objects.requireNonNull(customerName, "customerName required"); //
validation before construction
            return new OrderRequest(this);
        }
    }
}

// Usage - self-documenting, order-independent, impossible to swap email/phone by position:
OrderRequest request = OrderRequest.builder()
    .customerName("Ravi Kumar").email("ravi@example.com").phone("9876543210")
    .amount(new BigDecimal("499.00")).paymentType(PaymentType.CARD)
    .build();
```

Internal Working

- Making the target class's constructor **private** and routing all construction through `Builder.build()` guarantees an `OrderRequest` can never exist in a half-initialized or invalid state — `build()` is the single validation checkpoint (Book 04's fail-fast philosophy).
- **Lombok's @Builder** (widely used in Spring Boot codebases, Book 12) generates exactly this boilerplate automatically from field annotations — knowing the manual pattern is what makes `@Builder`-generated code fully legible rather than "magic."
- Builder solves the exact readability/safety problem OPS's `processOrder` method signature had — named setter-style calls (`.email(...)` , `.phone(...)`) make swapped arguments a compile-visible mistake (wrong method name) rather than a silent same-type parameter swap.

Real-World Example

`StringBuilder`, `Stream.Builder` (Book 07), and countless Spring Boot DTO builder methods in production codebases all use this exact pattern to construct complex, immutable objects readably.

Interview Answer

"Builder replaces long, error-prone, positional constructor or method parameter lists with fluent, named method calls, making construction self-documenting and immune to same-type parameter swaps. The target object's constructor is made private so it can only be built through the Builder's `build()` method, which acts as a single validation checkpoint guaranteeing the object is never left in an invalid state. Lombok's `@Builder` annotation, used throughout Spring Boot DTOs, generates exactly this pattern automatically."

Cross Questions

- Q: Why is the target class's constructor made private in the Builder pattern? → A: To force all construction through the Builder's `build()` method, which validates required fields before allowing an instance to be created, preventing half-initialized objects.
- Q: What real bug does Builder prevent that a long parameter list doesn't? → A: Two same-type parameters (like `email` and `phone`, both `String`) being accidentally passed in swapped order — Builder's named methods make this a visible naming mistake instead of a silent, compiler-invisible swap.

Tricky Questions

- Q: Is Builder overkill for a class with only two constructor parameters? → A: Generally yes — Builder's value is proportional to parameter count and optionality; a two-parameter constructor is usually clearer as a plain constructor.

Coding Exercise

L1: Implement `OrderRequest.Builder` replacing OPS's telescoping method signature. **L2:** Add required-field validation inside `build()`. **L3:** Compare the manual Builder to an equivalent Lombok `@Builder`-annotated class. **L4 (Interview):** Explain Builder and the bug class it prevents. **L5 (Senior — OPS Refactor):** Fully replace OPS's `processOrder(type, name, email, phone, amount, paymentType, status)` signature with a builder-constructed `OrderRequest`. **L6 (Mastery):** Design a Builder for a class with 10+ optional fields and explain why Builder beats overloaded constructors here.

CHAPTER 5 — Prototype Pattern

Telugu Explanation

కొన్నిసార్లు ఒక object ని scratch నుండి create చేయడం (DB fetch, network call, expensive computation) చాలా costly అవుతుంది — దానికి బదులు ఇప్పటికే ఉన్న object ని **clone** చేయడం వేగంగా ఉంటుంది. **Prototype pattern** ఒక object ని copy చేసే standard mechanism ఇస్తుంది, దాని exact class ని caller తెలుసుకోవాల్సిన అవసరం లేకుండా.

Professional English Explanation

Sometimes creating an object from scratch (a DB fetch, a network call, an expensive computation) is costly — cloning an existing object is faster. The **Prototype pattern** provides a standard mechanism to copy an object without the caller needing to know its exact class.

Java Code — Prototype for an Expensive-to-Build Order Template

```
interface OrderTemplate extends Cloneable {
    OrderTemplate copy();
}

class StandardShippingOrderTemplate implements OrderTemplate {
    private final ShippingRules rules; // expensive to build -
    loaded from DB/config once
    private final List<String> defaultLineItems;

    StandardShippingOrderTemplate(ShippingRules rules) {
        this.rules = rules; // pretend this took a
    } // real DB round-trip

    public OrderTemplate copy() {
        StandardShippingOrderTemplate clone = new StandardShippingOrderTemplate(this.rules);
        // reuse expensive `rules`
        clone.defaultLineItems.addAll(this.defaultLineItems); // deep-copy the mutable
        // list - shallow copy bug risk otherwise
        return clone;
    }
}

// Usage: OrderTemplate baseTemplate = loadExpensiveTemplateFromDb(); // done ONCE
//         OrderTemplate order1 = baseTemplate.copy(); // fast, no DB call
//         OrderTemplate order2 = baseTemplate.copy(); // fast, no DB call
```

Internal Working

- The critical implementation detail is **deep vs shallow copy**: naively copying `this.defaultLineItems` by reference (shallow) would mean `order1` and `order2` share the *same* mutable `List` — modifying one would silently corrupt the other; the explicit `addAll()` above performs a deep copy of the mutable field to avoid this classic Prototype bug.

- Prototype is justified specifically when object construction cost (DB/network/computation) is high relative to copying cost — for cheap-to-construct objects, Prototype adds a `copy()` method for no real benefit over just calling `new`.
- Java's built-in `Object.clone()` / `Cloneable` is widely considered a flawed implementation of this pattern (shallow-copies by default, awkward checked-exception handling) — most production code, as shown above, implements an explicit `copy()` method instead of relying on `Object.clone()`.

Real-World Example

A reporting system that builds an expensive, DB-backed "report template" object once at startup, then clones it per report request rather than re-querying the database for shared configuration on every single report.

Interview Answer

"Prototype clones an existing object instead of constructing a new one from scratch, which is worthwhile when construction is expensive relative to copying. The implementation must carefully deep-copy any mutable fields — a shallow copy would leave clones sharing the same mutable state, causing one clone's changes to silently affect another. Java's built-in `Cloneable` / `Object.clone()` is generally avoided in practice in favor of an explicit `copy()` method, because `clone()` shallow-copies by default and has an awkward API."

Cross Questions

- Q: What bug does a shallow copy introduce in Prototype? → A: Cloned objects end up sharing references to the same mutable fields (like a `List`), so mutating one clone's field silently affects all other clones sharing that reference.
- Q: When is Prototype NOT worth using? → A: When constructing a new instance from scratch is already cheap — adding a `copy()` method provides no real benefit over `new` in that case.

Tricky Questions

- Q: Does Java's `Object.clone()` automatically deep-copy an object's fields? → A: No — it performs a shallow copy by default; deep-copying mutable fields requires explicit code, which is one reason most codebases prefer a hand-written `copy()` method over relying on `clone()`.

Coding Exercise

L1: Implement `StandardShippingOrderTemplate.copy()` with correct deep-copying of its mutable list. **L2:** Deliberately introduce a shallow copy bug, demonstrate the shared-mutation problem, then fix it. **L3:** Compare using `Object.clone()` / `Cloneable` vs an explicit `copy()` method for the same class. **L4 (Interview):** Explain Prototype and the deep-vs-shallow copy pitfall. **L5 (Senior):** Identify one place in a hypothetical system where Prototype would meaningfully reduce cost, and justify it with numbers (construction cost vs copy cost). **L6 (Mastery):** Design a Prototype-based caching layer for expensive-to-build configuration objects.

CHAPTER 6 — Adapter Pattern

Telugu Explanation

OPS ఒక కొత్త third-party SMS gateway (LegacySmsApi) వాడాలంటే, దాని API (sendMessage(String to, String body)) మన Notifier interface (send(message, recipient)) తో match అవ్వదు. **Adapter pattern** ఒక incompatible interface ని expected interface గా "translate" చేస్తుంది — existing code ని మార్చాల్సిన అవసరం లేకుండా.

Professional English Explanation

If OPS needs to use a new third-party SMS gateway (LegacySmsApi), its API (sendMessage(String to, String body)) doesn't match our Notifier interface (send(message, recipient)). The **Adapter pattern** "translates" an incompatible interface into the expected one — without changing any existing code.

Java Code — Adapter Wrapping an Incompatible Third-Party API

```
class LegacySmsApi {                                // third-party class - CANNOT
be modified
    void sendMessage(String to, String body) { /* vendor's own implementation */ }
}

class LegacySmsAdapter implements Notifier {         // implements OUR interface
(Ch.2)
    private final LegacySmsApi legacyApi;

    LegacySmsAdapter(LegacySmsApi legacyApi) { this.legacyApi = legacyApi; }

    @Override
    public void send(String message, String recipient) { // translates OUR method
call...
        legacyApi.sendMessage(recipient, message);      // ...into THEIR method
call, with args reordered correctly
    }
}

// Usage: Notifier notifier = new LegacySmsAdapter(new LegacySmsApi());
// notifier.send("Order shipped!", "9876543210"); // OPS's code never knows LegacySmsApi
exists
```

Internal Working

- The Adapter **implements the interface the client already expects** (Notifier , from Ch.2's Factory Method) and internally **delegates** to the incompatible third-party object — this is composition, not inheritance, which is why Adapter works even for final third-party classes that can't be subclassed.
- This is exactly how many real Spring/JDBC integrations work: Spring's JdbcTemplate (Book 09) internally adapts each database vendor's specific Driver / Connection quirks behind one consistent API surface, so application code never deals with vendor-specific method signatures directly.

- Adapter adds a translation layer with **zero business logic** — its only job is reconciling method signatures/argument order/naming; if it starts containing real logic, it has quietly become something else (often a Facade, Ch.8).

Real-World Example

`java.util.Arrays.asList(array)` acts as an Adapter, wrapping a plain array so it can be used wherever the `List` interface is expected, without converting or copying the underlying array.

Interview Answer

"Adapter wraps an incompatible interface so it satisfies the interface the client code already expects, translating method calls (renaming, reordering arguments) via delegation rather than inheritance — which is why it works even with third-party classes that can't be modified or subclassed. Spring's `JdbcTemplate` conceptually plays this role for different JDBC drivers, giving application code one consistent API regardless of the underlying database vendor's specific `Driver` implementation."

Cross Questions

- Q: Why does Adapter use composition (wrapping) instead of inheritance? → A: The incompatible class is often a third-party or `final` class that can't be extended; composition works regardless, by holding a reference and delegating calls.
- Q: What should an Adapter NOT contain? → A: Real business logic — its sole responsibility is translating one interface's calls into another's; logic beyond that signals the class has become something else, like a Facade.

Tricky Questions

- Q: If `LegacySmsApi`'s method were later given the exact same signature as `Notifier.send`, would the Adapter still serve a purpose? → A: Only trivially (a pass-through) — Adapter's value specifically comes from reconciling a genuine mismatch; once the mismatch disappears, the wrapper adds no benefit and could be removed.

Coding Exercise

L1: Implement `LegacySmsAdapter` wrapping `LegacySmsApi` to satisfy `Notifier`. **L2:** Add a second legacy API with a differently-shaped method and adapt it too. **L3:** Confirm OPS's calling code required zero changes to use the new adapted provider. **L4 (Interview):** Explain Adapter and why it uses composition over inheritance. **L5 (Senior):** Identify a real interface mismatch in a hypothetical integration and design its Adapter. **L6 (Mastery):** Explain how `JdbcTemplate` (Book 09) conceptually adapts vendor-specific JDBC drivers.

CHAPTER 7 — Decorator Pattern

Telugu Explanation

OPS లో notification పంపే ముందు logging add చేయాలంటే, ప్రతి `Notifier` implementation లో logging code duplicate చేయాలి, లేదా inheritance తో ప్రతి combination కి ఒక subclass (LoggingEmailNotifier, LoggingSmsNotifier, RetryingLoggingEmailNotifier...) create చేయాలి — ఇది class explosion కి దారితీస్తుంది. **Decorator pattern** behavior ని runtime లో, wrapping ద్వారా, dynamically add చేస్తుంది.

Professional English Explanation

If OPS needs logging added before sending a notification, either every `Notifier` implementation duplicates logging code, or inheritance creates one subclass per combination (LoggingEmailNotifier, LoggingSmsNotifier, RetryingLoggingEmailNotifier...) — a class explosion. **Decorator pattern** adds behavior dynamically at runtime, via wrapping, instead.

Java Code — Decorator Adding Cross-Cutting Behavior

```
abstract class NotifierDecorator implements Notifier {           // implements the SAME
    interface it wraps
        protected final Notifier wrapped;
        NotifierDecorator(Notifier wrapped) { this.wrapped = wrapped; }
}

class LoggingNotifierDecorator extends NotifierDecorator {
    LoggingNotifierDecorator(Notifier wrapped) { super(wrapped); }
    public void send(String message, String recipient) {
        System.out.println("Sending to " + recipient + ": " + message);    // added behavior
        wrapped.send(message, recipient);                                // delegate to the
        wrapped object
    }
}

class RetryingNotifierDecorator extends NotifierDecorator {
    RetryingNotifierDecorator(Notifier wrapped) { super(wrapped); }
    public void send(String message, String recipient) {
        try { wrapped.send(message, recipient); }
        catch (Exception e) { wrapped.send(message, recipient); }        // one retry -
        added behavior, no subclass needed
    }
}

// Composable at runtime - stack decorators in any combination, no class explosion:
Notifier notifier = new LoggingNotifierDecorator(
    new RetryingNotifierDecorator(
        new EmailNotifier());
logging(retrying(email))
notifier.send("Order confirmed", "ravi@example.com");
```

Internal Working

- Each decorator **implements the same interface it wraps** (`Notifier`) and **delegates** to the wrapped instance after/before adding its own behavior — this lets decorators be **stacked in any**

combination and order at runtime, which is exactly what avoids the

`LoggingEmailNotifier / RetryingLoggingEmailNotifier` subclass-per-combination explosion.

- **JDK connection:** `java.io`'s `BufferedReader(new InputStreamReader(new FileInputStream(...)))` (Book 09-adjacent I/O knowledge) is the textbook JDK example of stacked Decorators — each layer adds one capability (buffering, character decoding) around the base stream.
- **Spring Security connection:** Spring's servlet filter chain (Book 10, Ch.4; Book 14, Ch.1) conceptually decorates the request-handling pipeline — each filter wraps the next, adding one cross-cutting concern (CORS, CSRF, auth) without any filter needing to know about the others.

Real-World Example

Coffee-shop ordering systems are the canonical Decorator teaching example (`new Milk(new Sugar(new Espresso()))`), but the exact same structure is what Java's I/O stream classes and Spring's filter chain use in real production code.

Interview Answer

"Decorator adds behavior to an object dynamically at runtime by wrapping it in another object implementing the same interface, which delegates to the wrapped instance while adding its own behavior before/after. This avoids the class explosion that inheritance would cause when many optional behaviors can combine in any order. Java's I/O stream classes (`BufferedReader` wrapping `InputStreamReader`) and Spring's servlet filter chain both use this exact structural pattern in production."

Cross Questions

- Q: Why does Decorator avoid the subclass-per-combination explosion inheritance would cause? → A: Decorators can be stacked and composed at runtime in any order and combination, rather than needing a separate compiled subclass for every possible combination of behaviors.
- Q: How does the Spring filter chain resemble Decorator? → A: Each filter wraps the next step in the chain, adding one concern without needing to know about other filters — structurally the same delegate-and-add-behavior pattern.

Tricky Questions

- Q: Does the order in which decorators are stacked matter? → A: Yes, often significantly — `LoggingNotifierDecorator(RetryingNotifierDecorator(email))` logs once per outer call, while `RetryingNotifierDecorator(LoggingNotifierDecorator(email))` would log on every retry attempt; the composition order changes observable behavior.

Coding Exercise

L1: Implement `LoggingNotifierDecorator` and `RetryingNotifierDecorator`. **L2:** Stack both decorators in two different orders and observe the behavioral difference. **L3:** Add a third decorator (e.g., rate-limiting) and compose all three. **L4 (Interview):** Explain Decorator and the class-explosion problem it avoids. **L5 (Senior — OPS Refactor):** Replace any inline logging/retry logic in OPS's

notification sending with composed decorators. **L6 (Mastery):** Explain the JDK I/O stream classes as a real-world Decorator chain, naming each layer's added responsibility.

CHAPTER 8 — Facade Pattern

Telugu Explanation

OPS లో `processOrder` ఒక్కటే method validation, payment, inventory, notification, logging అన్నింటినీ directly handle చేస్తుంది — ఇది ఒక complex subsystem కి caller ని directly expose చేస్తుంది. **Facade pattern** ఈ complex subsystem కి ఒక simple, unified interface ఇస్తుంది — internal complexity ని దాచిపెడుతూ.

Professional English Explanation

OPS's single `processOrder` method directly handles validation, payment, inventory, notification, and logging — exposing the caller directly to a complex subsystem. **Facade pattern** provides a simple, unified interface to that complex subsystem — hiding internal complexity.

Java Code — Facade Simplifying OPS's Complex Subsystem

```
class OrderProcessingFacade {                                // ONE simple entry point
    private final OrderValidator validator;                  // Ch.3's Abstract Factory
    private final PaymentGateway paymentGateway;             // Ch.3's Abstract Factory
    product
    private final InventoryService inventoryService;         // Ch.2's Factory Method
    private final Notifier notifier;                         // Ch.2's Factory Method
    product

    OrderResult placeOrder(OrderRequest request) {           // Ch.4's Builder-
        constructed request
        validator.validate(request);                         // subsystem call 1 -
        caller doesn't see this
        PaymentResult payment = paymentGateway.charge(request.amount()); // subsystem call 2
        inventoryService.reserve(request.items());            // subsystem call 3
        notifier.send("Order confirmed", request.email());    // subsystem call 4
        return new OrderResult(payment, OrderStatus.CONFIRMED); // caller sees ONE
        simple call, ONE simple result
    }
}

// Client code: OrderResult result = facade.placeOrder(request); // that's it - 4 subsystems
// hidden behind 1 call
```

Internal Working

- Facade does **not** eliminate the underlying complexity or the individual subsystem classes (`OrderValidator`, `PaymentGateway`, etc. all still exist and can still be used directly if needed) — it simply provides an additional, simpler entry point for the common case, which is the key distinction from Facade "replacing" a subsystem.
- **Spring Boot connection:** a `@Service` class in a typical layered architecture (Book 10, Ch.10) is very often acting as a Facade over multiple `@Repository` /client classes — `OrderService.placeOrder()` in a real Spring Boot app looks almost exactly like `OrderProcessingFacade.placeOrder()` above.

- Facade differs from Adapter (Ch.6): Adapter makes an **incompatible single interface compatible**; Facade makes a **complex multi-class subsystem simple** — they solve different problems even though both "wrap" something.

Real-World Example

A `CheckoutService` in an e-commerce backend typically facades calls to Cart, Pricing, Tax, Payment, and Shipping subsystems behind one `checkout()` method — exactly matching OPS's refactored shape here.

Interview Answer

"Facade provides one simple, unified interface over a complex subsystem of multiple classes, hiding coordination complexity from the caller without eliminating or replacing the underlying subsystem classes — they remain independently usable. In Spring Boot, a typical `@Service` class coordinating multiple repositories and clients is functioning as a Facade. It differs from Adapter, which reconciles one incompatible interface, rather than simplifying a multi-class subsystem."

Cross Questions

- Q: Does Facade eliminate the subsystem classes it wraps? → A: No — they remain fully usable independently; Facade just adds a simpler entry point for the common, coordinated use case.
- Q: How does Facade differ from Adapter? → A: Adapter reconciles one incompatible interface into an expected one; Facade simplifies access to a complex, multi-class subsystem — different problems despite both being "wrapping" patterns.

Tricky Questions

- Q: Is a Spring `@Service` class automatically a Facade? → A: Not automatically — it's a Facade specifically when it coordinates multiple underlying subsystems (repositories, clients) behind one simpler method; a `@Service` with no coordination logic is not meaningfully applying this pattern.

Coding Exercise

L1: Implement `OrderProcessingFacade.placeOrder()` coordinating 4 subsystem calls. **L2:** Confirm each subsystem class remains independently usable outside the facade. **L3:** Identify a Spring `@Service` class (real or hypothetical) already acting as a Facade. **L4 (Interview):** Explain Facade and how it differs from Adapter. **L5 (Senior — OPS Refactor):** Refactor OPS's monolithic `processOrder` method into a Facade coordinating the by-now-refactored validator/payment/inventory/notifier subsystems. **L6 (Mastery):** Design a Facade for a checkout flow spanning 5 subsystems (cart, pricing, tax, payment, shipping).

CHAPTER 9 — Proxy Pattern

Telugu Explanation

ఇది Book 11, Ch.7's AOP proxy మరియు Book 13's `@Transactional` మెకానిజం directly ఆధారపడిన pattern. **Proxy** ఒక object కి బదులుగా నిలబడి, real object కి call forward చేసే ముందు/తర్వాత అదనపు logic (access control, lazy loading, logging, caching) add చేస్తుంది — real object తనకు తాను దాని గురించి తెలుసుకోవాల్సిన అవసరం లేకుండా.

Professional English Explanation

This is the pattern Book 11, Ch.7's AOP proxies and Book 13's `@Transactional` mechanism are directly built on. A **Proxy** stands in for an object, adding extra logic (access control, lazy loading, logging, caching) before/after forwarding the call to the real object — without the real object needing any awareness of it.

Java Code — Proxy Adding Access Control Without Modifying the Real Object

```
interface PaymentGateway { PaymentResult charge(BigDecimal amount); }

class RealPaymentGateway implements PaymentGateway {           // the real object - no
security-check code inside it
    public PaymentResult charge(BigDecimal amount) { /* actual charging logic */ return new
PaymentResult(true); }
}

class SecurePaymentGatewayProxy implements PaymentGateway {     // same interface as the
real object
    private final PaymentGateway realGateway;
    private final SecurityContext securityContext;               // Book 14's
authenticated user context

    SecurePaymentGatewayProxy(PaymentGateway realGateway, SecurityContext ctx) {
        this.realGateway = realGateway; this.securityContext = ctx;
    }

    public PaymentResult charge(BigDecimal amount) {
        if (!securityContext.hasRole("PAYMENT_INITIATOR")) {    // added check - the
real object knows nothing of this
            throw new AccessDeniedException("Not authorized to charge payments");
        }
        return realGateway.charge(amount);                      // forward to the real
object only if authorized
    }
}
```

Internal Working

- The Proxy implements the **exact same interface** as the real object, so callers are unaware whether they're holding a proxy or the real thing — this transparency is what makes Proxy composable with everything else (a caller using `PaymentGateway` doesn't know or care).

- **This IS how Book 11, Ch.7's Spring AOP and Book 13, Ch.7's `@Transactional` actually work under the hood** — Spring generates a runtime proxy around your `@Service` / `@Repository` bean; calling a `@Transactional` method actually calls the *proxy* first, which opens a transaction, then forwards to your real method, then commits/rolls back — exactly the shape shown above, generated automatically instead of hand-written.
- **Book 16, Ch.7's Resilience4j Circuit Breaker** is the same mechanism again: `@CircuitBreaker` wraps your real method call in a proxy that intercepts, checks circuit state, and either forwards the call or fails fast/falls back — three separate books' "magic annotations" are all this one pattern.

Real-World Example

Hibernate's lazy-loaded entity associations (Book 13) return a **proxy object** in place of the real related entity — the proxy only triggers the actual database query when a real field is first accessed, which is precisely why accessing a lazy association outside an open persistence context throws `LazyInitializationException`.

Interview Answer

"Proxy stands in for a real object behind the same interface, adding logic like access control, lazy loading, or logging before/after forwarding to the real object, which remains unaware of the proxy's existence. This is exactly the mechanism behind Spring AOP, `@Transactional` (Book 13), and Resilience4j's `@CircuitBreaker` (Book 16) — all three generate a runtime proxy around your bean that intercepts the method call to add their respective cross-cutting behavior before delegating to your actual code. Hibernate's lazy-loading also uses proxy objects, which is why accessing a lazy field outside an open session throws `LazyInitializationException`."

Deep Interview Answer (Senior/Architect)

"A senior-level detail worth stating precisely: Spring's proxies are why `@Transactional` / `@CircuitBreaker` / `@Cacheable` all silently fail to apply on **self-invocation** — calling `this.chargePayment()` from within the same bean bypasses the proxy entirely, since the call never goes back out through the container-managed proxy object; only calls that arrive from *outside* the bean (through the proxy Spring actually injected elsewhere) get the added behavior. This single fact explains a huge fraction of 'why isn't my `@Transactional` working' production bugs, and connects directly back to Book 11 Ch.7 and Book 13 Ch.7's identical caveat."

Cross Questions

- Q: What three annotations across Books 11/13/16 are all implemented using this same Proxy pattern? → A: Spring AOP's general proxying, `@Transactional` (Book 13), and Resilience4j's `@CircuitBreaker` (Book 16) — all generate a runtime proxy that intercepts the call before forwarding to the real method.
- Q: Why does calling an annotated method via `this.method()` from inside the same class bypass its proxy-added behavior? → A: The call never passes back out through the container-managed proxy object — it's a direct in-object call, so the proxy's interception logic (transaction, circuit breaker check, etc.) never runs.

Tricky Questions

- Q: Does the real object (e.g., `RealPaymentGateway`) need to contain any code aware of the proxy wrapping it? → A: No — this transparency is the whole point of the pattern; the real object's code is completely unaware of and unaffected by whatever proxy sits in front of it.

Coding Exercise

L1: Implement `SecurePaymentGatewayProxy` wrapping `RealPaymentGateway` with an access check.

L2: Add a logging proxy layer and stack it with the security proxy (compare to Ch.7's Decorator — note the conceptual overlap). **L3:** Reproduce a self-invocation bug: call a `@Transactional` method via `this.` from inside the same Spring bean and observe the transaction not applying. **L4**

(Interview): Explain Proxy and name the three Spring/Resilience4j annotations built on it. **L5**

(Senior): Explain, from memory, exactly why self-invocation bypasses proxy-added behavior. **L6**

(Mastery): Design a caching proxy for an expensive read-only service call, explaining cache invalidation strategy.

CHAPTER 10 — Composite Pattern

Telugu Explanation

ఒక order లో individual `LineItem` లు మరియు "bundles" (multiple items కలిపి ఒక్క discount తో అమ్మే product group) రెండూ ఉండొచ్చు, మరియు bundle లో మళ్ళీ bundles ఉండొచ్చు (nested). **Composite pattern** individual objects మరియు వాటి compositions ని **ఒకే uniform interface** తో treat చేయడానికి వీలు కల్పిస్తుంది — client code individual item ని handle చేస్తుందా, whole tree ని handle చేస్తుందా అని తేడా చూపించాల్సిన అవసరం లేదు.

Professional English Explanation

An order may contain individual `LineItem`s and "bundles" (groups of items sold together at a discount), and bundles can nest within bundles. **Composite pattern** lets individual objects and compositions of them be treated through **one uniform interface** — client code doesn't need to distinguish handling a single item from handling a whole tree.

Java Code — Composite for Nested Order Line Items

```
interface OrderComponent {                                // uniform interface for both
leaf and composite
    BigDecimal getPrice();
}

class LineItem implements OrderComponent {                // LEAF - a single product
    private final BigDecimal price;
    LineItem(BigDecimal price) { this.price = price; }
    public BigDecimal getPrice() { return price; }
}

class ProductBundle implements OrderComponent {           // COMPOSITE - contains other
OrderComponents
    private final List<OrderComponent> components = new ArrayList<>();
    private final BigDecimal discountPercent;

    void add(OrderComponent component) { components.add(component); } // can add LineItems OR
nested ProductBundles

    public BigDecimal getPrice() {
        BigDecimal total = components.stream()
            .map(OrderComponent::getPrice)                // Book 07 - method
reference + Stream
            .reduce(BigDecimal.ZERO, BigDecimal::add);    // recursion
happens naturally here -
        return total.multiply(BigDecimal.ONE.subtract(discountPercent)); // a nested
bundle's getPrice() calls ITS children's getPrice()
    }
}

// Usage - client code treats a single item and a deeply nested bundle IDENTICALLY:
OrderComponent laptop = new LineItem(new BigDecimal("50000"));
OrderComponent bundle = new ProductBundle(new BigDecimal("0.10")); // 10% off
bundle.add(laptop);
bundle.add(new LineItem(new BigDecimal("1500")));           // mouse
BigDecimal total = bundle.getPrice();                       // works the same
whether `bundle` is nested 5 levels deep or not
```

Internal Working

- `ProductBundle.getPrice()` recursively calls `getPrice()` on each of its children — if a child is itself a `ProductBundle`, this naturally recurses arbitrarily deep without any special-case code; this is the elegance Composite provides over manually checking "is this a leaf or a group?" everywhere in client code.
- Both `LineItem` (leaf) and `ProductBundle` (composite) implement the exact same `OrderComponent` interface — this is what allows `bundle.add(laptop)` and `bundle.add(nestedBundle)` to use the identical method signature.
- Composite is the right pattern whenever a domain naturally forms a **tree** — file systems (files and folders), UI component trees (a `Panel` containing `Buttons` and nested `Panel`s), and organizational hierarchies are classic Composite use cases beyond this order example.

Real-World Example

A file system's `du` (disk usage) command effectively uses Composite logic — a file's size is a leaf value, and a directory's size is the recursive sum of its contents, whether those contents are files or further nested directories.

Interview Answer

"Composite lets individual objects (leaves) and compositions of them (composites) be treated through the same interface, so client code doesn't need to special-case 'is this one item or a group?' A composite's operation typically recurses into its children naturally, which is what lets it handle arbitrarily deep nesting (a bundle containing bundles containing bundles) without extra logic. It's the right fit whenever a domain naturally forms a tree structure — file systems, UI component trees, and nested product bundles are classic examples."

Cross Questions

- Q: Why do both `LineItem` and `ProductBundle` implement the same `OrderComponent` interface?
→ A: So client code can treat a single item and an arbitrarily nested group of items identically, calling the same method (`getPrice()`) without checking which kind it is.
- Q: How does Composite handle arbitrarily deep nesting without special-case code? → A: A composite's method (like `getPrice()`) recursively invokes the same method on each child — if a child is itself a composite, this naturally recurses to any depth.

Tricky Questions

- Q: Is Composite useful for a domain that is NOT naturally tree-shaped (e.g., a flat list of unrelated objects)? → A: No — its value comes specifically from unifying leaf/group handling in a recursive tree structure; forcing it onto flat, non-hierarchical data adds unnecessary abstraction.

Coding Exercise

L1: Implement `LineItem` and `ProductBundle` with a working `getPrice()` . **L2:** Nest a `ProductBundle` inside another `ProductBundle` and confirm price calculation recurses correctly. **L3:** Add a `getItemCount()` operation to `OrderComponent` and implement it for both leaf and composite. **L4 (Interview):** Explain Composite and why it fits naturally tree-shaped domains. **L5 (Senior):** Design a Composite structure for a UI component tree (Panel/Button/nested Panel). **L6 (Mastery):** Explain why Composite would be a poor fit for OPS's flat list of order line items if bundling were never a requirement.

CHAPTER 11 — Observer Pattern

Telugu Explanation

OPS లో order status change అయినప్పుడు నేరుగా notification code call చేసాము — ఇది OrderProcessor ని Notifier తో tightly couple చేస్తుంది. **Observer pattern** ఒక "subject" (Order) state change అయినప్పుడు, తనకు subscribe అయిన అన్ని "observers" (Notification, Analytics, Audit) కి automatically inform చేస్తుంది — subject తన observers ఎవరో, ఎన్ని ఉన్నాయో తెలుసుకోవాల్సిన అవసరం లేకుండా. ఇది Book 17's Kafka pub-sub కి direct పూర్వగామి.

Professional English Explanation

OPS directly called notification code when order status changed — tightly coupling OrderProcessor to Notifier. The **Observer pattern** has a "subject" (Order) automatically inform all subscribed "observers" (Notification, Analytics, Audit) when its state changes — without the subject needing to know who or how many observers exist. This is the direct in-process ancestor of Book 17's Kafka pub-sub.

Java Code — Observer Decoupling Order State Changes from Reactions

```
interface OrderObserver { void onStatusChanged(Order order, OrderStatus newStatus); }

class Order { // the SUBJECT
    private final List<OrderObserver> observers = new ArrayList<>();
    private OrderStatus status;

    void addObserver(OrderObserver observer) { observers.add(observer); } // subject knows
    only the interface, not concrete types

    void setStatus(OrderStatus newStatus) {
        this.status = newStatus;
        for (OrderObserver observer : observers) { // notify ALL
            observer.onStatusChanged(this, newStatus);
        }
    }
}

class NotificationObserver implements OrderObserver {
    public void onStatusChanged(Order order, OrderStatus newStatus) { /* send email/SMS */ }
}

class AnalyticsObserver implements OrderObserver {
    public void onStatusChanged(Order order, OrderStatus newStatus) { /* update dashboard */ }
}

// Usage: order.addObserver(new NotificationObserver()); order.addObserver(new
AnalyticsObserver());
// order.setStatus(OrderStatus.SHIPPED); // BOTH observers react - Order class has zero
knowledge of either concrete class
```

Internal Working

- `Order` depends only on the `OrderObserver` **interface** — adding a third observer (`FraudDetectionObserver`) requires **zero changes** to the `Order` class, exactly mirroring Book 16, Ch.1's pub-sub advantage, but in-process rather than across a network/message broker.
- **This is precisely Kafka's conceptual origin (Book 17):** a Kafka topic is Observer's "subject" scaled to a distributed system — producers publish (`setStatus`), and any number of independent consumer groups (`observers`) react, without the producer knowing they exist. Understanding Observer in-process is what makes Kafka's pub-sub model immediately intuitive rather than a brand-new idea.
- **Spring's `ApplicationEventPublisher`** (Book 11-adjacent Spring infrastructure) is Spring's built-in, framework-managed implementation of exactly this pattern — `publisher.publishEvent(new OrderStatusChangedEvent(...))` and any number of `@EventListener`-annotated methods react, with Spring managing the observer registration instead of a manual `List<OrderObserver>`.

Real-World Example

A GUI framework's button-click listeners are a textbook Observer implementation — a `Button` (subject) doesn't know or care how many `ActionListener`s (observers) are registered, or what any of them do when clicked.

Interview Answer

"Observer lets a subject automatically notify all subscribed observers when its state changes, without the subject knowing their concrete types or count — it depends only on an observer interface. This is the direct in-process ancestor of Kafka's pub-sub model (Book 17): a topic is Observer's 'subject' scaled across a distributed system, where producers publish without knowing which consumer groups exist. Spring's `ApplicationEventPublisher` and `@EventListener` are the framework-managed, in-process implementation of this exact same pattern."

Cross Questions

- Q: What has to change in the `Order` class to add a third observer? → A: Nothing — `Order` only depends on the `OrderObserver` interface, so any new implementation can subscribe without any change to the subject.
- Q: How does Observer relate conceptually to Kafka's pub-sub model (Book 17)? → A: Kafka scales the exact same idea across a distributed system — a topic is the subject, consumer groups are observers, and the producer doesn't need to know how many or which consumers exist.

Tricky Questions

- Q: In the code shown, if `NotificationObserver.onStatusChanged` throws an exception, what happens to `AnalyticsObserver`? → A: It depends on the loop's implementation — as written, an uncaught exception from one observer would halt the loop and prevent later observers from being notified; production Observer implementations typically catch and log per-observer exceptions to avoid one faulty observer blocking the rest.

Coding Exercise

L1: Implement `Order` as a subject with `NotificationObserver` and `AnalyticsObserver`. **L2:** Add a third observer with zero changes to the `Order` class. **L3:** Fix the exception-propagation issue so one failing observer doesn't block the others. **L4 (Interview):** Explain Observer and its relationship to Kafka pub-sub. **L5 (Senior — OPS Refactor):** Replace OPS's direct notification call with the Observer pattern. **L6 (Mastery):** Implement the same behavior using Spring's `ApplicationEventPublisher` / `@EventListener` and compare it to the manual Observer implementation.

CHAPTER 12 — Strategy Pattern

Telugu Explanation

OPS లో `if (paymentType.equals("CARD")) ... else if (paymentType.equals("UPI")) ...` ఒక algorithm-selection if-else chain — కొత్త payment method add చేయాలంటే ఈ block ని modify చేయాలి.

Strategy pattern ఒక్కో algorithm ని దాని సొంత class లో encapsulate చేసి, వాటిని runtime లో interchangeable గా చేస్తుంది.

Professional English Explanation

OPS's `if (paymentType.equals("CARD")) ... else if (paymentType.equals("UPI")) ...` is an algorithm-selection if-else chain — adding a new payment method requires modifying this block.

Strategy pattern encapsulates each algorithm in its own class, making them interchangeable at runtime.

Java Code — Strategy Replacing OPS's Payment If-Else

```
interface PaymentStrategy { PaymentResult pay(BigDecimal amount); }

class CardPaymentStrategy implements PaymentStrategy {
    public PaymentResult pay(BigDecimal amount) { /* card charging logic */ return new
PaymentResult(true); }
}
class UpiPaymentStrategy implements PaymentStrategy {
    public PaymentResult pay(BigDecimal amount) { /* UPI charging logic */ return new
PaymentResult(true); }
}

class OrderProcessor {
    private PaymentStrategy paymentStrategy; // held as an interface
reference - swappable

    void setPaymentStrategy(PaymentStrategy strategy) { this.paymentStrategy = strategy; } //
chosen/injected at runtime

    PaymentResult checkout(BigDecimal amount) {
        return paymentStrategy.pay(amount); // OrderProcessor has
ZERO if-else about payment type
    }
}

// Usage: processor.setPaymentStrategy(new UpiPaymentStrategy()); processor.checkout(new
BigDecimal("499"));
// Adding a new payment method = adding a new class, NOT modifying OrderProcessor.
```

Internal Working

- `OrderProcessor` holds a `PaymentStrategy` **reference**, not a concrete type — this is composition-over-inheritance and dependency inversion (Book 02) in direct action: behavior is injected, not hardcoded.

- **Comparison to `Comparator<T>` (Book 05):** `Collections.sort(list, comparator)` is literally Strategy — the sorting *algorithm's structure* stays fixed, but the *comparison logic* is a pluggable strategy object, exactly like `PaymentStrategy` above.
- **Strategy vs Ch.15's State pattern** (a common point of confusion): Strategy is chosen once and used per-call by the *client's* explicit choice; State pattern (Ch.15) changes *automatically* based on the *object's own* internal condition — structurally near-identical code, fundamentally different intent.

Real-World Example

Spring Security's `PasswordEncoder` interface (Book 14) is a Strategy — `BCryptPasswordEncoder`, `Pbkdf2PasswordEncoder`, etc., are interchangeable algorithm implementations selected by configuration, with the rest of the authentication code depending only on the `PasswordEncoder` interface.

Interview Answer

"Strategy encapsulates each algorithm variant in its own class implementing a common interface, and the client holds a reference to that interface rather than a concrete implementation — making algorithms swappable at runtime and adding a new one require only a new class, not modifying existing dispatch logic. `Comparator<T>` (Book 05) is Strategy applied to sorting, and Spring Security's `PasswordEncoder` (Book 14) is Strategy applied to hashing algorithms. It's often confused with State (Ch.15) since the code shape is similar, but Strategy is client-chosen per call, while State changes automatically based on the object's own internal condition."

Cross Questions

- Q: What does adding a new payment method require under the Strategy refactor, versus OPS's original if-else? → A: Only a new class implementing `PaymentStrategy` — `OrderProcessor` itself needs zero modification, unlike the original if-else chain.
- Q: How is `Comparator<T>` (Book 05) an example of Strategy? → A: `Collections.sort()`'s algorithm structure stays fixed, but the actual comparison logic is a pluggable, interchangeable `Comparator` implementation passed in — exactly Strategy's shape.

Tricky Questions

- Q: Is Strategy the same as Factory Method (Ch.2)? → A: No — Factory Method is about *creating* an object; Strategy is about *choosing swappable behavior/algorithms* to use, often after the object already exists — they're frequently used together (a factory creates the chosen strategy) but solve different problems.

Coding Exercise

L1: Implement `CardPaymentStrategy` and `UpiPaymentStrategy` and inject one into `OrderProcessor`. **L2:** Add a third payment strategy with zero changes to `OrderProcessor`. **L3:** Implement a custom `Comparator` and connect its structure explicitly to Strategy. **L4 (Interview):** Explain Strategy and how it differs from State (preview Ch.15). **L5 (Senior — OPS Refactor):** Fully

remove OPS's payment-type if-else chain using Strategy. **L6 (Mastery):** Explain why Spring Security's `PasswordEncoder` is a production Strategy implementation.

CHAPTER 13 — Template Method Pattern

Telugu Explanation

Order processing లో steps (validate → charge → reserve inventory → notify) ఎప్పుడూ ఒకే **sequence** లో జరుగుతాయి, కానీ ప్రతి order type (Standard, Express, International) కి కొన్ని steps వేరుగా ఉండొచ్చు.

Template Method algorithm యొక్క **skeleton/sequence** ని ఒక base class లో fix చేసి, individual steps ని subclasses override చేయడానికి వదిలేస్తుంది.

Professional English Explanation

Order processing's steps (validate → charge → reserve inventory → notify) always happen in the same **sequence**, but each order type (Standard, Express, International) may implement some steps differently. **Template Method** fixes the algorithm's **skeleton/sequence** in a base class, leaving individual steps for subclasses to override.

Java Code — Template Method Fixing the Sequence, Varying the Steps

```
abstract class OrderProcessingTemplate {

    final OrderResult process(OrderRequest request) {                // final - the SEQUENCE
    cannot be changed by subclasses
        validate(request);
        PaymentResult payment = charge(request);
        reserveInventory(request);
        sendConfirmation(request);                                // this step is the SAME
    for every subclass
        return new OrderResult(payment, OrderStatus.CONFIRMED);
    }

    abstract void validate(OrderRequest request);                    // subclasses MUST
    implement
    abstract PaymentResult charge(OrderRequest request);
    abstract void reserveInventory(OrderRequest request);

    private void sendConfirmation(OrderRequest request) {            // shared step - not
    overridable
        System.out.println("Confirmation sent to " + request.email());
    }
}

class ExpressOrderProcessor extends OrderProcessingTemplate {
    void validate(OrderRequest r) { /* express-specific validation, e.g., address must support
    express */ }
    PaymentResult charge(OrderRequest r) { /* charge with express surcharge */ return new
    PaymentResult(true); }
    void reserveInventory(OrderRequest r) { /* reserve from express-priority warehouse */ }
}
// Usage: new ExpressOrderProcessor().process(request);    // the SEQUENCE is fixed; only these
3 steps varied
```

Internal Working

- Making `process()` `final` is deliberate and important — it prevents subclasses from reordering or skipping steps, which is the entire point: the *sequence* is a fixed contract, only individual *steps* vary — this is the "Hollywood Principle" ("don't call us, we'll call you") in action.
- **Spring connection:** `JdbcTemplate` (Book 09) and `RestTemplate` / `RestClient` 's internal request-execution flow are named "Template" for exactly this reason — they fix the skeleton (open connection/execute/handle exceptions/close connection) while letting callers supply the varying part (the actual SQL, or a `RowMapper`) via a callback — Spring's Template classes are literal, direct applications of this GoF pattern, hence the shared name.
- Template Method vs Strategy (Ch.12): Template Method varies **steps within a fixed algorithm structure** via inheritance; Strategy swaps the **entire algorithm** via composition — Template Method is "customize parts of one process," Strategy is "choose between whole alternative processes."

Real-World Example

JUnit 5's test lifecycle (Book 15) — `@BeforeEach` → test method → `@AfterEach` — is a Template Method: the framework fixes the sequence, and each test class supplies the varying parts (the actual setup/test/teardown code).

Interview Answer

"Template Method fixes an algorithm's overall sequence in a base class's `final` method, while deferring individual steps to abstract methods subclasses must implement — preventing subclasses from altering the sequence itself, only the step details. Spring's `JdbcTemplate` and `RestTemplate` are directly named after this pattern: they fix the skeleton (connection handling, exception translation) while callers supply the varying logic via a callback. It differs from Strategy (Ch.12), which swaps an entire algorithm via composition, rather than customizing steps within one fixed sequence via inheritance."

Cross Questions

- Q: Why is `process()` declared `final` in the Template Method base class? → A: To prevent subclasses from reordering or skipping steps — the sequence itself is meant to be a fixed contract; only the individual step implementations vary.
- Q: How does `JdbcTemplate` (Book 09) embody this pattern? → A: It fixes the skeleton of connection acquisition, statement execution, exception translation, and resource cleanup, while the caller supplies the varying SQL/ `RowMapper` — exactly Template Method's structure, which is why it's named "Template."

Tricky Questions

- Q: Could Strategy (Ch.12) be used instead of Template Method here? → A: Not quite equivalently — Strategy would let you swap the entire payment/validation/etc. algorithm wholesale via composition, but wouldn't enforce that all order types share the exact same fixed step sequence;

Template Method's inheritance-based structure is what guarantees the sequence itself can't be altered.

Coding Exercise

L1: Implement `OrderProcessingTemplate` with the fixed `process()` sequence. **L2:** Implement `ExpressOrderProcessor` and `InternationalOrderProcessor` with differing step logic. **L3:** Attempt to override `process()` itself and confirm the compiler prevents it (`final`). **L4 (Interview):** Explain Template Method and how `final` enforces its contract. **L5 (Senior — OPS Refactor):** Refactor OPS's order-type-specific inline logic into Template Method subclasses. **L6 (Mastery):** Explain precisely how `JdbcTemplate` 's callback-based API embodies this pattern.

CHAPTER 14 — Command Pattern

Telugu Explanation

Order operations (place, cancel, refund) ని objects గా encapsulate చేయాలంటే — ఉదా: queue లో store చేయడానికి, undo చేయడానికి, లేదా async గా execute చేయడానికి (Book 08's `Runnable`) — **Command pattern** ఒక request ని ఒక్క object గా wrap చేస్తుంది, దాన్ని parameter గా pass చేయడానికి, queue చేయడానికి, log చేయడానికి, లేదా undo చేయడానికి వీలు కల్పిస్తూ.

Professional English Explanation

To encapsulate order operations (place, cancel, refund) as objects — for example, to queue them, undo them, or execute them asynchronously (Book 08's `Runnable`) — the **Command pattern** wraps a request as an object, enabling it to be passed as a parameter, queued, logged, or undone.

Java Code — Command Encapsulating Order Operations with Undo Support

```
interface OrderCommand {
    void execute();
    void undo();
}

class CancelOrderCommand implements OrderCommand {
    private final Order order;
    private OrderStatus previousStatus;           // remembered for undo

    CancelOrderCommand(Order order) { this.order = order; }

    public void execute() {
        this.previousStatus = order.getStatus();   // capture state
        BEFORE changing it
        order.setStatus(OrderStatus.CANCELLED);   // Ch.11's Observer
        fires here too
    }

    public void undo() { order.setStatus(previousStatus); } // restore captured
    state
}

class CommandInvoker {                          // decouples "what to
    run" from "how/when to run it"
    private final Deque<OrderCommand> history = new ArrayDeque<>(); // Book 05 - Deque as
    an undo stack

    void executeCommand(OrderCommand command) {
        command.execute();
        history.push(command);
    }

    void undoLast() {
        if (!history.isEmpty()) history.pop().undo(); // Book 08's
        ExecutorService can run commands too:
    }
    executorService.submit(() -> command.execute());
}
```

Internal Working

- Wrapping the operation as an `OrderCommand` object (rather than calling `order.cancel()` directly) is what enables the **undo stack** — a plain method call has no natural way to be "reversed" later, but a `Command` object can capture whatever prior state it needs at execute-time and restore it on `undo()`.
- Book 08 connection:** a `Runnable / Callable` submitted to an `ExecutorService` **is** the Command pattern — the task is encapsulated as an object, decoupled entirely from the thread pool that eventually executes it; understanding Command explains why `Runnable` is designed as an interface with a single `run()` method rather than executors just accepting a raw method reference with side effects tracked elsewhere.
- Commands compose naturally with **Ch.11's Observer**: `execute()` above triggers `order.setStatus(...)`, which fires all of `Order`'s registered observers — Command and

Observer frequently appear together in real systems without conflict.

Real-World Example

A text editor's undo/redo stack is the textbook Command implementation — every user action (type, delete, format) is captured as a command object with its own `undo()`, pushed onto a history stack exactly like `CommandInvoker` above.

Interview Answer

"Command encapsulates a request or operation as an object with an `execute()` method (and often `undo()`), decoupling what should be done from when or how it's invoked. This enables queuing, logging, and undo functionality that a plain direct method call can't naturally support, since the command object can capture whatever state it needs at execution time to reverse itself later. Book 08's `Runnable / Callable` submitted to an `ExecutorService` is a direct real-world application of Command — the task is a self-contained object, fully decoupled from the thread pool executing it."

Cross Questions

- Q: Why can't a plain direct method call (`order.cancel()`) easily support undo the way a Command object can? → A: A Command object can capture whatever prior state it needs at execution time and restore it in its own `undo()` method; a bare method call has no natural place to store that captured state for later reversal.
- Q: How is `Runnable` an example of Command? → A: It encapsulates a unit of work as an object with a single `run()` method, fully decoupled from whichever thread/executor eventually invokes it — exactly Command's structure.

Tricky Questions

- Q: Does every Command implementation need to support `undo()`? → A: No — undo is a common but optional extension; many Command uses (like `Runnable` submitted for async execution) only need `execute()` / `run()`, with no reversal concept at all.

Coding Exercise

L1: Implement `CancelOrderCommand` with working `execute()` / `undo()`. **L2:** Implement `CommandInvoker` with an undo stack and test undoing multiple commands in order. **L3:** Submit an `OrderCommand`'s `execute()` via an `ExecutorService` (Book 08) and explain the Command-pattern connection. **L4 (Interview):** Explain Command and how it enables undo functionality. **L5 (Senior — OPS Refactor):** Convert OPS's direct cancel/refund operations into `OrderCommand` implementations with undo support. **L6 (Mastery):** Design a full command-queue-based order processing system with logging, retry, and undo.

CHAPTER 15 — State Pattern

Telugu Explanation

OPS లో `if (status.equals("PLACED")) ... else if (status.equals("SHIPPED")) ...` ఒక status-based if-else chain — కొత్త status add చేయాలంటే modify చేయాలి, మరియు invalid transitions (CANCELLED నుండి SHIPPED కి వెళ్ళడం లాంటివి) prevent చేయడం కష్టం. **State pattern** ప్రతి status ని దాని సొంత class గా encapsulate చేసి, valid transitions ని ఆ class లోనే enforce చేస్తుంది. ఇది Book 16, Ch.8's SAGA order-status transitions కి exact object-oriented foundation.

Professional English Explanation

OPS's `if (status.equals("PLACED")) ... else if (status.equals("SHIPPED")) ...` is a status-based if-else chain — adding a new status requires modification, and preventing invalid transitions (like CANCELLED jumping to SHIPPED) is hard to enforce. **State pattern** encapsulates each status as its own class, enforcing valid transitions within that class itself. This is the exact object-oriented foundation underneath Book 16, Ch.8's SAGA order-status transitions.

Java Code — State Replacing OPS's Status If-Else with Enforced Transitions

```
interface OrderState {
    OrderState ship(); // returns the NEXT state,
    // or throws if invalid
    OrderState cancel();
}

class PlacedState implements OrderState {
    public OrderState ship() { return new ShippedState(); } // valid: PLACED -> SHIPPED
    public OrderState cancel() { return new CancelledState(); } // valid: PLACED -> CANCELLED
}

class ShippedState implements OrderState {
    public OrderState ship() { throw new IllegalStateException("Already shipped"); }
    public OrderState cancel() { throw new IllegalStateException("Cannot cancel a shipped order"); } // enforced HERE
}

class CancelledState implements OrderState {
    public OrderState ship() { throw new IllegalStateException("Cannot ship a cancelled order"); }
    public OrderState cancel() { throw new IllegalStateException("Already cancelled"); }
}

class Order {
    private OrderState state = new PlacedState(); // Order delegates
    // transition logic entirely
    void ship() { this.state = state.ship(); }
    void cancel() { this.state = state.cancel(); }
}
```


Internal Working

- Each state class knows **only its own valid transitions** — `Order` itself contains zero transition-validation logic; it just delegates to whatever the current state object allows, which is precisely what makes adding a new status (`ReturnedState`) a matter of adding one new class, not touching every existing if-else branch.
- **Book 16, Ch.8 connection:** the SAGA pattern's order-status transitions (PENDING → CONFIRMED/CANCELLED, with compensating actions) are conceptually State pattern applied across a distributed system — State pattern here is the same idea in a single JVM/object, and understanding it makes SAGA's state-machine diagrams immediately readable as "just Ch.15's pattern, distributed."
- State (this chapter) vs Strategy (Ch.12): the code shape is nearly identical (an interface with interchangeable implementations), but the *trigger* differs — Strategy is chosen explicitly by client code; State transitions happen **as a side effect of the object's own behavior** (`state.ship()` returning a new state), which the object then adopts automatically.

Real-World Example

A traffic light controller is a classic State pattern application — `RedState` , `YellowState` , `GreenState` each know only their own valid next state, and the controller never needs an external if-else chain to decide what "next" means.

Interview Answer

"State encapsulates each state as its own class implementing a common interface, with each state class responsible for producing the valid next state (or rejecting invalid transitions) for each triggering action. The containing object (`Order`) delegates entirely to its current state object rather than maintaining if-else transition logic itself, so adding a new state means adding one new class. This is conceptually the same idea underlying Book 16, Ch.8's SAGA order-status state machine, just distributed across microservices instead of contained in one object. It's often confused with Strategy (Ch.12) due to similar code shape, but State transitions automatically as a consequence of the object's behavior, while Strategy is explicitly chosen by the client."

Cross Questions

- Q: What prevents an invalid transition like CANCELLED → SHIPPED in this implementation? → A: `CancelledState.ship()` explicitly throws — the invalid transition is impossible to reach because each state class only exposes transitions it considers valid.
- Q: How does State relate to Book 16, Ch.8's SAGA pattern? → A: SAGA's order-status transitions are the same State-pattern idea applied across a distributed system with compensating actions, rather than contained within one object's state field.

Tricky Questions

- Q: If `Order.ship()` and `Order.cancel()` looked identical in code shape to Ch.12's Strategy-based `checkout()` , how would you tell an interviewer these are different patterns? → A: By what triggers the swap — State's next-state object is produced automatically as part of handling the

action (`state.ship()` returns the new state), while Strategy's implementation is explicitly selected and set by client code beforehand, independent of any single method call's outcome.

Coding Exercise

L1: Implement `PlacedState`, `ShippedState`, `CancelledState` with enforced valid/invalid transitions. **L2:** Add a `ReturnedState` reachable only from `ShippedState`. **L3:** Attempt an invalid transition and confirm it throws rather than silently succeeding. **L4 (Interview):** Explain State and how it differs from Strategy despite similar code shape. **L5 (Senior — OPS Refactor):** Fully remove OPS's status if-else chain using State. **L6 (Mastery):** Map this chapter's State machine explicitly onto Book 16, Ch.8's SAGA order-status diagram, state by state.

CHAPTER 16 — Chain of Responsibility Pattern

Telugu Explanation

OPS లో validation, authentication, logging అన్నీ ఒకే method లో inline గా ఉన్నాయి. **Chain of Responsibility** ఒక request ని handlers యొక్క ఒక chain వెంట పంపుతుంది — ప్రతి handler దాన్ని handle చేయాలో, తర్వాత handler కి pass చేయాలో నిర్ణయిస్తుంది. ఇది Book 10/14's servlet filter chain మరియు Book 16's API Gateway filters కి **direct, exact precursor**.

Professional English Explanation

OPS had validation, authentication, and logging all inline in one method. **Chain of Responsibility** sends a request along a chain of handlers — each handler decides whether to handle it or pass it to the next. This is the **direct, exact precursor** to Book 10/14's servlet filter chain and Book 16's API Gateway filters.

Java Code — Chain of Responsibility Replacing OPS's Inline Validation/Logging

```
abstract class OrderHandler {
    protected OrderHandler next; // reference to the NEXT
    handler in the chain
    OrderHandler setNext(OrderHandler next) { this.next = next; return next; } // fluent
    chain-building

    abstract void handle(OrderRequest request);

    protected void passToNext(OrderRequest request) {
        if (next != null) next.handle(request); // pass along - or the
        chain simply ends here
    }
}

class ValidationHandler extends OrderHandler {
    void handle(OrderRequest request) {
        if (request.amount().compareTo(BigDecimal.ZERO) <= 0)
            throw new IllegalArgumentException("Invalid amount"); // this handler can
        STOP the chain
        passToNext(request); // or let it
        continue
    }
}

class LoggingHandler extends OrderHandler {
    void handle(OrderRequest request) {
        System.out.println("Processing order for " + request.customerName());
        passToNext(request);
    }
}

class FraudCheckHandler extends OrderHandler {
    void handle(OrderRequest request) {
        if (isSuspicious(request)) throw new SecurityException("Flagged for review");
        passToNext(request);
    }
}

// Building and using the chain - order of construction IS the order of execution:
OrderHandler chain = new ValidationHandler();
chain.setNext(new LoggingHandler()).setNext(new FraudCheckHandler());
chain.handle(orderRequest); // request flows
through all 3, in order
```

Internal Working

- Each handler holds only a reference to the **next** handler, not the whole chain — this is what lets handlers be composed, reordered, or have new ones inserted **without any handler needing to know about the others**, exactly the property Book 10, Ch.4's servlet filters and Book 14, Ch.1's Spring Security filter chain rely on.
- **This IS, structurally, exactly Book 14, Ch.1's Security Filter Chain and Book 16, Ch.6's API Gateway filter chain** — CorsFilter → CsrfFilter → AuthFilter → AuthorizationFilter (Book 14) and Gateway's addRequestHeader → circuitBreaker filters (Book 16) are both literal, real-

world Chain of Responsibility implementations; this chapter's hand-written version is what those "magic" framework filter chains are built from underneath.

- Any handler in the chain can **short-circuit** it (as `ValidationHandler` does by throwing instead of calling `passToNext`) — this maps directly to how a Spring Security filter can reject a request (401/403) before it ever reaches later filters or the controller.

Real-World Example

Java's own Servlet `FilterChain.doFilter()` API is a literal, JDK-level Chain of Responsibility implementation — each `Filter` receives the chain and explicitly decides whether to call `chain.doFilter()` to continue, or stop by not calling it (e.g., after writing an error response).

Interview Answer

"Chain of Responsibility passes a request along a sequence of handlers, each deciding independently whether to process it and/or pass it to the next handler — handlers only know about the next link, not the whole chain, which lets them be composed, reordered, or extended independently. This is precisely the structure behind Book 14's Spring Security filter chain and Book 16's API Gateway filter chain — both are real, production Chain of Responsibility implementations, not just conceptually similar. Any handler can short-circuit the chain, exactly like a Security filter rejecting a request with 401/403 before it ever reaches the controller."

Cross Questions

- Q: What does each handler in the chain need to know about the other handlers? → A: Only a reference to the next handler — nothing about the full chain's composition, which is what makes handlers independently reorderable/insertable.
- Q: Name two real Spring/production filter chains that are literal Chain of Responsibility implementations. → A: Spring Security's filter chain (Book 14, Ch.1) and Spring Cloud Gateway's per-route filter chain (Book 16, Ch.6).

Tricky Questions

- Q: If `ValidationHandler` throws instead of calling `passToNext`, do `LoggingHandler` and `FraudCheckHandler` still run? → A: No — throwing (or simply not calling `passToNext`) stops the chain right there; this short-circuiting is a deliberate, expected part of the pattern, exactly like a Security filter rejecting a request before authorization or the controller are ever reached.

Coding Exercise

L1: Implement `ValidationHandler`, `LoggingHandler`, `FraudCheckHandler` and chain them together. **L2:** Reorder the chain (fraud check before logging) and confirm no handler class needed modification. **L3:** Add a handler that short-circuits the chain under a specific condition and verify later handlers don't run. **L4 (Interview):** Explain Chain of Responsibility and name its real Spring Security/Gateway implementations. **L5 (Senior — OPS Refactor):** Fully replace OPS's inline validation/logging/fraud logic with a composed handler chain. **L6 (Mastery):** Explain, chapter by

chapter, how OPS has now been fully refactored using all 16 patterns — trace each remaining piece of the original god-class to the pattern that replaced it.

FINAL REVISION NOTES

- Creational patterns (Ch.1–5) control *how* objects are created — Singleton (one instance), Factory Method (one product via a method), Abstract Factory (a family of products), Builder (step-by-step construction), Prototype (cloning).
 - Structural patterns (Ch.6–10) control how objects are *composed* — Adapter (interface translation), Decorator (dynamic behavior stacking), Facade (simplified subsystem access), Proxy (transparent stand-in with added behavior), Composite (uniform leaf/tree handling).
 - Behavioral patterns (Ch.11–16) control how objects *interact* — Observer (automatic notification), Strategy (swappable algorithms), Template Method (fixed sequence, varying steps), Command (operations as objects), State (self-managed transitions), Chain of Responsibility (sequential handler delegation).
 - Nearly every pattern in this book is already running inside Spring/Spring Boot/Kafka: Singleton (bean scope), Factory (`BeanFactory`), Proxy (AOP/ `@Transactional` / `@CircuitBreaker`), Template Method (`JdbcTemplate` / `RestTemplate`), Observer (`ApplicationEventPublisher` /Kafka), Strategy (`PasswordEncoder` / `Comparator`), Chain of Responsibility (Security filter chain/Gateway filters).
 - Every pattern has a cost (added indirection/classes) — the OPS refactor exercises deliberately show applying each pattern to a REAL problem it solves, not applying it just because it exists.
-



CHEAT SHEET

Pattern	One-Line Summary	Spring/Java Real-World Match
Singleton	Exactly one instance, global access	Spring bean default scope
Factory Method	Delegate creation of one product	BeanFactory / ApplicationContext
Abstract Factory	Create a family of related products	Profile-scoped @Configuration classes
Builder	Step-by-step, named construction	Lombok @Builder , DTO builders
Prototype	Clone instead of construct	Expensive-template caching
Adapter	Translate an incompatible interface	JDBC driver abstraction
Decorator	Stack behavior dynamically	Java I/O streams, filter chains
Facade	Simplify a complex subsystem	Layered @Service classes
Proxy	Transparent stand-in, added behavior	AOP, @Transactional , @CircuitBreaker
Composite	Uniform leaf/tree handling	UI trees, nested bundles
Observer	Auto-notify on state change	ApplicationEventPublisher , Kafka pub-sub
Strategy	Swap algorithms via composition	Comparator , PasswordEncoder
Template Method	Fixed sequence, varying steps	JdbcTemplate , RestTemplate , JUnit lifecycle
Command	Operation as an object	Runnable / Callable , undo stacks
State	Self-managed valid transitions	SAGA order-status machine (Book 16)
Chain of Responsibility	Sequential handler delegation	Security filter chain, Gateway filters



INTERVIEW QUESTION BANK — Design Patterns

Beginner

1. What is Singleton, and why is the enum-based implementation preferred?
2. What is the difference between Factory Method and Abstract Factory?
3. What problem does Builder solve?

Intermediate 4. Explain Adapter vs Facade — both "wrap" something, but how do they differ? 5. Explain Decorator and how it avoids inheritance's class explosion. 6. Explain Observer and its relationship to pub-sub/Kafka. 7. Explain Strategy and give a JDK example.

Advanced 8. Explain Proxy and name three Spring/Resilience4j annotations built on it. 9. Explain why self-invocation bypasses Spring's proxy-based `@Transactional`. 10. Explain Template Method and how `JdbcTemplate` embodies it. 11. Explain the difference between State and Strategy despite similar code shape. 12. Explain Chain of Responsibility and how Spring Security's filter chain implements it literally.

Senior/Architect 13. Given a real god-class, identify which 4–5 patterns would most improve it and justify the order of refactoring. 14. Explain precisely where Kafka's Observer-like pub-sub model diverges from in-process Observer (delivery guarantees, ordering, replay). 15. Design a resilient payment-processing pipeline combining Strategy (provider selection), Proxy (circuit breaking), and Command (retry/undo).



CROSS-QUESTION ENGINE — Sample Chains

- Q: Why does Proxy explain `@Transactional` 's self-invocation limitation? → A: Spring generates a proxy around the bean; only calls arriving from outside the bean pass through that proxy, so `this.method()` bypasses the transactional behavior entirely. → Cross: Does the same limitation apply to `@CircuitBreaker` (Book 16, Ch.7)? → A: Yes — it's the identical proxy mechanism, so self-invocation bypasses circuit-breaking too.
 - Q: How does Observer differ from Kafka's pub-sub (Book 17)? → A: Same core idea, but Kafka adds durability (persisted log), replay, and distributed delivery guarantees Observer's in-memory list of listeners doesn't have. → Cross: What in-process Spring mechanism sits between the two? → A: `ApplicationEventPublisher` / `@EventListener` — framework-managed Observer, still in-process, no persistence or replay.
-



CONSOLIDATED EXERCISES (All Levels)

- Fully refactor `OrderProcessingSystem` (OPS) end-to-end using all 16 patterns from this book's chapters.
 - Implement a Spring Boot mini-service where you can point to at least 5 of these patterns already at work in the framework itself (Singleton bean scope, Factory `ApplicationContext` , Proxy `@Transactional` , Template `JdbcTemplate` , Observer `@EventListener`).
 - Design pattern selection exercise: given 5 different bad-design code smells, name the correct pattern for each and justify why an alternative pattern would be worse.
-



ONE-DAY REVISION PLAN (≈6 hours)

Time	Focus
0:00–1:15	Ch.1–5: Creational patterns
1:15–2:30	Ch.6–10: Structural patterns
2:30–4:00	Ch.11–13: Observer, Strategy, Template Method
4:00–5:00	Ch.14–16: Command, State, Chain of Responsibility
5:00–6:00	Full interview bank + trace OPS's complete refactor end-to-end



ONE-WEEK MASTER REVISION PLAN

Day	Focus
1	Ch.1-3 — Singleton, Factory Method, Abstract Factory
2	Ch.4-5 — Builder, Prototype
3	Ch.6-8 — Adapter, Decorator, Facade
4	Ch.9-10 — Proxy (deep focus — Spring connections), Composite
5	Ch.11-12 — Observer, Strategy
6	Ch.13-16 — Template Method, Command, State, Chain of Responsibility
7	Full OPS refactor end-to-end + mock interview using the question bank



FINAL MASTERY CHECKLIST

- ☐ I can implement all 5 Creational patterns and explain when each is justified.
- ☐ I can implement all 5 Structural patterns and explain when each is justified.
- ☐ I can implement all 6 Behavioral patterns and explain when each is justified.
- ☐ I can name the Spring/Spring Boot/Kafka mechanism built on each pattern, without hesitation.
- ☐ I can explain precisely why self-invocation bypasses Proxy-based Spring annotations.
- ☐ I can distinguish Strategy from State, and Adapter from Facade, under interview pressure.
- ☐ I fully refactored `OrderProcessingSystem` using all 16 patterns.
- ☐ I can identify over-engineering — recognizing when NOT to apply a pattern.

Next: `19_Low_Level_Design.md` — Book 19, applying these exact patterns to complete, interview-standard LLD case studies (Parking Lot, BookMyShow, Elevator, and more).